

PORTFOLIO

Aufgabenstellung zum Kurs: Machine Learning Systems Design (DLMDSPMLSD01_D)

INHALTSVERZEICHNIS

1.	AUFGABENSTELLUNG	2
1.1.	Aufgabe 1: Pinguinklassifizierer	2
1.1.1.	Konzeptionsphase	2
1.1.2.	Erarbeitungs-/Reflexionsphase.....	3
1.1.3.	Finalisierungsphase.....	4
1.2.	Aufgabe 2: Dokumentzusammenfasser	4
1.2.1.	Konzeptionsphase	5
1.2.2.	Erarbeitungs-/Reflexionsphase.....	6
1.2.3.	Finalisierungsphase.....	6
1.3.	Aufgabe 3: eigene Aufgabenstellung	7
1.3.1.	Konzeptionsphase	7
1.3.2.	Erarbeitungs-/Reflexionsphase.....	8
1.3.3.	Finalisierungsphase.....	8
2.	BETREUUNGSPROZESS	9
3.	ZUSATZINFORMATIONEN ZUR BEWERTUNG	9
4.	FORMALIA UND VORGABEN ZUR ABGABE	10
4.1.	Bestandteile der Prüfungsleistung	10
4.2.	Formalia zur Abgabe digitaler Dateien	11
4.3.	Formalia für das Abstract	12

1. AUFGABENSTELLUNG

Im Rahmen dieses Portfoliokurses ist eine der folgenden Aufgabenstellungen zu wählen.

Hinweis zum Urheberrecht und zur Plagiatsprüfung:

Es wird darauf hingewiesen, dass der IU Internationale Hochschule GmbH das Urheberrecht der Prüfungsaufgaben/Aufgabenstellungen obliegt. Einer Veröffentlichung der Aufgabenstellungen auf Drittplattformen wird ausdrücklich widersprochen. Im Falle einer Zuwiderhandlung stehen der Hochschule u.a. Unterlassungsansprüche zu. Zudem weisen wir darauf hin, dass jede eingereichte schriftliche Ausarbeitung mittels einer Plagiatssoftware überprüft wird. Wir empfehlen daher auch, keinesfalls ausgearbeitete Lösungen zu teilen, da dies den Verdacht eines Plagiates begründen kann.

1.1. Aufgabe 1: Pinguinklassifizierer

Dein:e Kolleg:in ist auf einer Antarktisexpedition, bei der das Verhalten und die Evolution von Pinguinen analysiert werden soll. Erst vor Ort stellen sie fest, dass die Excel-basierte Speicherung und Analyse neuer Daten (d. h. Daten neu untersuchter Pinguine) zu umständlich wäre. Das Team benötigt eine Softwarelösung, die den Prozess der Pinguinbestimmung vereinfacht. Du wirst gebeten, diese Software zu erstellen. Aus Gründen der Machbarkeit innerhalb des Kurses/des Zeitkontingents beschränken wir uns auf einen Teil der relevanten Maßnahmen. Grundsätzlich ist es Pflicht, bei der Umsetzung verschiedene Aspekte zu bedenken. Es soll ein Klassifizierungsalgorithmus trainiert und in einer nutzerfreundlichen Software integriert werden. Die Daten des untersuchten Pinguins sollen mit den Trainingsdaten abgeglichen und visuell dargestellt werden. Dabei ist zu bedenken, dass in dem Antarktisteam zwar Biolog:innen, aber keine Informatiker:innen anwesend sind und deswegen eine stabile Lösung benötigt wird, die unabhängig vom Betriebssystem in Betrieb genommen werden kann.

Die Maßnahme: Erstelle eine Software zur Pinguinklassifizierung.

Inhaltlich ist **Folgendes Pflicht:**

- plattformunabhängige Bereitstellung der Software,
- UI zur Dateneingabe,
- Klassifizierungsalgorithmus,
- Beschreibung der Modellperformanz,
- visuelle Darstellung des neuen Datenpunkts in Relation zu den „alten“ Daten und
- Speicherung der neuen Daten.

1.1.1. Konzeptionsphase

Diese Phase stellt den wichtigsten Teil des Prozesses dar. Alles, was in dieser Phase übersehen oder vergessen wird, wirkt sich später negativ auf die Umsetzung aus und führt im schlimmsten Fall zu unbrauchbaren Ergebnissen.

Der erste Schritt besteht darin, ein schriftliches Konzept zu erstellen, in dem alles beschrieben wird, was zum Programmieren des Backends und Frontends der Software und ihrer Bereitstellung gehört. **Dieser Schritt ist vielleicht der wichtigste des gesamten Prozesses.** Es ist entscheidend, sich für diese Phase genügend Zeit zu nehmen, bevor die nächsten Schritte unternommen werden können. Das Konzept kann basierend auf folgenden Schritten erstellt werden:

- **Datengrundlage:** Es wird empfohlen, den „Palmer Pinguin“-Datensatz als Trainingsdatensatz zu verwenden. Dieser ist in verschiedenen Datenrepositories leicht zu finden.
- **Technologische Werkzeuge:** Erwäge, Softwaretools wie R in Kombination mit RShiny oder Python mit Dash zu verwenden. Alternativ kannst Du andere vergleichbare Python-Frameworks evaluieren, die für die Erstellung interaktiver Webanwendungen geeignet sind.
- **Softwarearchitektur:** Entwickle eine umfassende Software-Architektur mithilfe von UML-Sequenzdiagrammen, um die Interaktionen und Abläufe innerhalb der Anwendung präzise abzubilden.
- **Benutzeroberflächendesign:** Erstelle ein vorläufiges Design der Benutzeroberfläche mittels einer schematischen Zeichnung, um die Funktionalität und Einfachheit der Dateneingabe zu gewährleisten.
- **Bereitstellung und Distribution:** Plane strategische Maßnahmen zur effizienten Bereitstellung und Verteilung der Software, um eine einfache und konsistente Installation über verschiedene Plattformen hinweg sicherzustellen.
- **Skalierbarkeit und Anpassungsfähigkeit:** Entwickle Konzepte zur Skalierung der Software und zur Anpassung des Modells bei Verfügbarkeit neuer Daten, um kontinuierliche Verbesserungen und Aktualisierungen zu unterstützen.
- **Datenschutz und Sicherheit:** Berücksichtige Datenschutzrichtlinien, Umsetzungsstrategien zur Sicherstellung von Datenintegrität, Vermeidung von Systemfehlern, Wahrung der Datenrelevanz sowie der IT-Sicherheit im gesamten Entwicklungsprozess.

Für die Abgabe ist ein **konzeptioneller Text (ca. 2 DIN-A4-Seiten)** zu erstellen und durch Grafiken zu ergänzen, sodass sämtliche Überlegungen beinhaltet sind. Das Konzept wird als **PDF-Datei** via pebblePad eingereicht.

Während des gesamten Prozesses gibt es im Rahmen der Online-Veranstaltungen die Möglichkeit, über Ideen und Entwürfe zu sprechen und sich Feedback einzuholen. In den Online-Veranstaltungen werden exemplarisch Arbeiten besprochen, die den Tutor:innen zuvor übermittelt wurden. Hier besteht für alle die Möglichkeit, sich einzubringen und vom Feedback der anderen zu lernen. **Es wird empfohlen, diese Kanäle zu nutzen, um Fehler zu vermeiden und Verbesserungen vorzunehmen. Erst danach sollen die Ergebnisse in der ersten Phase abgegeben werden.** Hier erfolgt ein abschließendes Feedback durch die Tutor:innen und die Arbeit in der zweiten Phase kann beginnen.

1.1.2. Erarbeitungs-/Reflexionsphase

In dieser Phase erfolgt die Implementierung der Klassifizierungssoftware basierend auf dem in der Konzeptionsphase entwickelten Plan unter Verwendung der ausgewählten Frameworks. Es ist zu erwarten, dass anfänglich Herausforderungen auftreten werden, die jedoch als integraler Bestandteil des Lernprozesses betrachtet werden sollten. Ausführliche Dokumentationen zu RShiny und Dash stehen auf den jeweiligen Dokumentationsseiten zur Verfügung und können hilfreich sein, ebenso wie zahlreiche Tutorials und Beispielprojekte, die zur Problemlösung beitragen können.

Das Frontend und das Backend soll programmiert, getestet und dokumentiert werden. Folgende Schritte sind hierzu umzusetzen:

- Richte ein **offenes GitHub-Repository** ein, um den Code unter Nutzung von Versionskontrollmechanismen zu verwalten und zu speichern.
- Dokumentiere und kommentiere den Code gemäß bewährter Praktiken der Softwareentwicklung, um Lesbarkeit und Wartbarkeit sicherzustellen.
- Verpacke den Code in eine Einheit zur standardisierten Verfügbarkeit, z. B. mithilfe von Container-Technologien.
- Führe eine Bewertung des trainierten Modells anhand standardmäßiger Leistungskennzahlen durch, um die Effizienz und Genauigkeit des Algorithmus zu quantifizieren.

Eine Erläuterung der Vorgehensweise wird als **Text eingereicht (1 DIN-A4-Seite)**. Die Datei soll den Link zum GitHub-Repository enthalten. Außerdem sollen die Schritte dieser Phase kurz beschrieben werden. Die Erklärung wird als **PDF-Datei** via pebblePad eingereicht.

Während des gesamten Prozesses gibt es im Rahmen der Online-Veranstaltungen und der weiteren Kanäle die Möglichkeit, über Ideen und Entwürfe zu sprechen und sich ausreichend Rückmeldung, Tipps und Hinweise zu holen. **Es wird empfohlen, diese Kanäle zu nutzen, um Fehler zu vermeiden und Verbesserungen vorzunehmen. Erst danach sollen die Ergebnisse in der zweiten Phase abgegeben werden.** Nach dem folgenden abschließenden Feedback durch die Tutor:innen wird in der dritten Phase an dem finalen Entwurf weitergearbeitet.

1.1.3. Finalisierungsphase

In der Abschlussphase besteht das Ziel darin, die Software auf Bugs zu prüfen und auf Stabilität zu testen. Bestimmte Elemente müssen eventuell noch einmal verbessert oder geändert werden. Du erstellst eine ausführliche **Zusammenfassung – full abstract (2 DIN-A4-Seiten)**, in der die Lösung der Aufgabe inhaltlich und konzeptionell beschrieben und der technische Ansatz kurz und übersichtlich dargestellt werden. Dabei sollte auch kurz auf die Qualität der Daten und die Ergebnisse der Analyse eingegangen werden. Diese Zusammenfassung sollte auch einen Ausblick darauf enthalten, wie das Verfahren weiter verbessert werden kann, sowie eine persönliche Reflexion über das Projekt: Was hast Du auf persönlicher Ebene gelernt? Auf welche Probleme bist Du gestoßen, und wie hast Du diese gelöst? Was sind Deine Problemlösungsstrategien für ähnliche anstehende Projekte?

Zusätzlich wird eine **Softwaredokumentation** geschrieben und ein **Walkthrough** aufgezeichnet. Die Video-Datei des Walkthroughs samt Dokumentation und dem **Container mit der Software** wird in einen **ZIP-Ordner** verpackt und hochgeladen.

Auch in der Finalisierungsphase gibt es im Rahmen der Online-Veranstaltungen und der weiteren Kanäle die Möglichkeit, sich ausreichend Rückmeldung, Tipps und Hinweise zu holen, **bevor** das fertige Produkt final abgegeben wird. **Es wird empfohlen, diese Kanäle zu nutzen, um Fehler zu vermeiden und Verbesserungen vorzunehmen.**

1.2. Aufgabe 2: Dokumentzusammenfasser

Du arbeitest in einem Patentanwaltsbüro, das mit sensiblen Daten umgeht. Die Patentanwälte müssen bei jeder Patentanmeldung zahlreiche Dokumente lesen und bewerten. Du wirst beauftragt, ein In-house-Tool zu programmieren, das PDF-Dokumente zusammenfassen soll. Die Anwälte haben keine technischen Vorkenntnisse im Umgang mit spezieller Software und benötigen deshalb eine intuitive und benutzerfreundliche Lösung.

Aus Gründen der Machbarkeit innerhalb des Zeitrahmens konzentrieren wir uns auf wesentliche Aspekte der Softwareentwicklung. Die Anwendung soll so gestaltet sein, dass sie einfach zu bedienen ist und dennoch eine zuverlässige Zusammenfassung der Inhalte liefert. Letztlich ist eine plattformunabhängige Bereitstellung unerlässlich, damit die Software auf verschiedenen Geräten eingesetzt werden kann, ohne dass technische Anpassungen erforderlich sind.

Die Maßnahme: Erstelle eine Software zur Dokumentzusammenfassung.

Inhaltlich ist Folgendes Pflicht:

- plattformunabhängige Bereitstellung der Software,
- UI zur PDF-Dateieingabe,
- Textzusammenfassung,

- Beschreibung der Modellperformanz und
- Ausgabe der Textzusammenfassung.

1.2.1. Konzeptionsphase

Diese Phase stellt den wichtigsten Teil des Prozesses dar. Alles, was in dieser Phase übersehen oder vergessen wird, wirkt sich später negativ auf die Umsetzung aus und führt im schlimmsten Fall zu unbrauchbaren Ergebnissen.

Der erste Schritt besteht darin, ein schriftliches Konzept zu erstellen, in dem alles beschrieben wird, was zum Programmieren des Backends und Frontends der Software und ihrer Bereitstellung gehört. **Dieser Schritt ist vielleicht der wichtigste des gesamten Prozesses.** Es ist entscheidend, sich für diese Phase genügend Zeit zu nehmen, bevor die nächsten Schritte unternommen werden können. Das Konzept kann basierend auf folgenden Schritten erstellt werden:

- **Datengrundlage:** Es wird empfohlen, den „cnn dailymail“-Datensatz zur Abschätzung der Modellperformanz bzw. für das Finetuning zu verwenden. Dieser ist in verschiedenen Datenrepositories, z. B. Huggingface, leicht zu finden.
- **Technologische Werkzeuge:** Nutze Softwaretools wie Python in Kombination mit Dash. Alternativ kannst Du andere vergleichbare Python-Frameworks evaluieren, die für die Erstellung interaktiver Webanwendungen geeignet sind.
- **Softwarearchitektur:** Entwickle eine umfassende Software-Architektur mithilfe von UML-Sequenzdiagrammen, um die Interaktionen und die Abläufe innerhalb der Anwendung präzise abzubilden.
- **Benutzeroberflächendesign:** Erstelle ein vorläufiges Design der Benutzeroberfläche mittels einer schematischen Zeichnung, um die Funktionalität und Einfachheit der Dateneingabe zu gewährleisten.
- **Bereitstellung und Distribution:** Plane strategische Maßnahmen zur effizienten Bereitstellung und Verteilung der Software, um eine einfache und konsistente Installation über verschiedene Plattformen hinweg sicherzustellen.
- **Skalierbarkeit und Anpassungsfähigkeit:** Entwickle Konzepte zur Skalierung der Software und zur Anpassung des Modells bei Verfügbarkeit neuer Daten, um kontinuierliche Verbesserungen und Aktualisierungen zu unterstützen.
- **Datenschutz und Sicherheit:** Berücksichtige Datenschutzrichtlinien, Umsetzungsstrategien zur Sicherstellung von Datenintegrität, Vermeidung von Systemfehlern, Wahrung der Datenrelevanz sowie der IT-Sicherheit im gesamten Entwicklungsprozess.

Für die Abgabe ist ein **konzeptioneller Text (ca. 2 DIN-A4-Seiten)** zu erstellen und durch Grafiken zu ergänzen, sodass sämtliche Überlegungen beinhaltet sind. Das Konzept wird als **PDF-Datei** via pebblePad eingereicht.

Während des gesamten Prozesses gibt es im Rahmen der Online-Veranstaltungen die Möglichkeit, über Ideen und Entwürfe zu sprechen und sich Feedback einzuholen. In den Online-Veranstaltungen werden exemplarisch Arbeiten besprochen, die den Tutor:innen zuvor übermittelt wurden. Hier besteht für alle die Möglichkeit, sich einzubringen und vom Feedback der anderen zu lernen. **Es wird empfohlen, diese Kanäle zu nutzen, um Fehler zu vermeiden und Verbesserungen vorzunehmen. Erst danach sollen die Ergebnisse in der ersten Phase abgegeben werden.** Hier erfolgt ein abschließendes Feedback durch die Tutor:innen und die Arbeit in der zweiten Phase kann beginnen.

1.2.2. Erarbeitungs-/Reflexionsphase

In dieser Phase erfolgt die Implementierung der Software basierend auf dem in der Konzeptionsphase entwickelten Plan unter Verwendung der ausgewählten Frameworks. Es ist zu erwarten, dass anfänglich Herausforderungen auftreten werden, die jedoch als integraler Bestandteil des Lernprozesses betrachtet werden sollten. Ausführliche Dokumentationen zu Dash stehen auf den jeweiligen Dokumentationsseiten zur Verfügung und können hilfreich sein, ebenso wie zahlreiche Tutorials und Beispielprojekte, die zur Problemlösung beitragen können.

Das Frontend und das Backend soll programmiert, getestet und dokumentiert werden. Folgende Schritte sind hierzu umzusetzen:

- Richte ein **offenes GitHub-Repository** ein, um den Code unter Nutzung von Versionskontrollmechanismen zu verwalten und zu speichern.
- Dokumentiere und kommentiere den Code gemäß bewährter Praktiken der Softwareentwicklung, um Lesbarkeit und Wartbarkeit sicherzustellen.
- Führe eine Bewertung des Modells zur Textzusammenfassung anhand standardmäßiger Leistungskennzahlen durch, um die Effizienz und Genauigkeit des Algorithmus zu quantifizieren.

Eine Erläuterung der Vorgehensweise wird als **Text eingereicht (1 DIN-A4-Seite)**. Die Datei soll den Link zum GitHub-Repository enthalten. Außerdem sollen die Schritte dieser Phase kurz beschrieben werden. Die Erklärung wird als **PDF-Datei** via pebblePad eingereicht.

Während des gesamten Prozesses gibt es im Rahmen der Online-Veranstaltungen und der weiteren Kanäle die Möglichkeit, über Ideen und Entwürfe zu sprechen und sich ausreichend Rückmeldung, Tipps und Hinweise zu holen. **Es wird empfohlen, diese Kanäle zu nutzen, um Fehler zu vermeiden und Verbesserungen vorzunehmen. Erst danach sollen die Ergebnisse in der zweiten Phase abgegeben werden.** Nach dem folgenden abschließenden Feedback durch die Tutor:innen wird in der dritten Phase an dem finalen Entwurf weitergearbeitet.

1.2.3. Finalisierungsphase

In der Abschlussphase besteht das Ziel darin, die Software auf Bugs zu prüfen und auf Stabilität zu testen. Bestimmte Elemente müssen eventuell noch einmal verbessert oder geändert werden. Du erstellst eine ausführliche **Zusammenfassung – full abstract (2 DIN-A4-Seiten)**, in der die Lösung der Aufgabe inhaltlich und konzeptionell beschrieben und der technische Ansatz kurz und übersichtlich dargestellt werden. Dabei sollte auch kurz auf die Qualität der Daten und die Ergebnisse der Analyse eingegangen werden. Diese Zusammenfassung sollte auch einen Ausblick darauf enthalten, wie das Verfahren weiter verbessert werden kann, sowie eine persönliche Reflexion über das Projekt: Was hast Du auf persönlicher Ebene gelernt? Auf welche Probleme bist Du gestoßen, und wie hast Du diese gelöst? Was sind Deine Problemlösungsstrategien für ähnliche anstehende Projekte?

Zusätzlich wird eine **Softwaredokumentation** geschrieben und ein **Walkthrough** aufgezeichnet. Die Video-Datei des Walkthroughs samt Dokumentation, die den Link zur **GitHub-Repository** enthält, wird in einen **ZIP-Ordner** verpackt und hochgeladen.

Auch in der Finalisierungsphase gibt es im Rahmen der Online-Veranstaltungen und der weiteren Kanäle die Möglichkeit, sich ausreichend Rückmeldung, Tipps und Hinweise zu holen, bevor das fertige Produkt final abgegeben wird. Es wird empfohlen, diese Kanäle zu nutzen, um Fehler zu vermeiden und Verbesserungen vorzunehmen.

1.3. Aufgabe 3: eigene Aufgabenstellung

In Absprache mit dem:der Tutor:in können eigene Themenvorschläge eingebracht werden, die im Umfang und in der Komplexität den zuvor beschriebenen Aufgaben ähneln. Die abgegebenen Dokumente sollen die gleichen Anforderungen wie bei Aufgabe 1 oder 2 erfüllen. Beispielsweise kann anstelle des Pinguinklassifizierers ein anderer Klassifizierer mit einem anderen Datensatz entwickelt werden, oder anstelle des vorgeschlagenen Zusammenfassungstools kann etwa ein Chatbot oder Textklassifizierer (z. B. zur Sentiment-Einschätzung) erstellt werden.

1.3.1. Konzeptionsphase

Diese Phase stellt den wichtigsten Teil des Prozesses dar. Alles, was in dieser Phase übersehen oder vergessen wird, wirkt sich später negativ auf die Umsetzung aus und führt im schlimmsten Fall zu unbrauchbaren Ergebnissen.

Der erste Schritt besteht darin, ein schriftliches Konzept zu erstellen, in dem alles beschrieben wird, was zum Programmieren des Backends und Frontends der Software und ihrer Bereitstellung gehört. **Dieser Schritt ist vielleicht der wichtigste des gesamten Prozesses.** Es ist entscheidend, sich für diese Phase genügend Zeit zu nehmen, bevor die nächsten Schritte unternommen werden können. Das Konzept kann basierend auf folgenden Schritten erstellt werden:

- **Datengrundlage:** Wähle einen Datensatz entsprechend Deiner Fragestellung. Der Datensatz sollte in einer öffentlichen Datenrepository, wie z. B. Kaggle, Huggingface, UCI Machine Learning Repository o. Ä. verfügbar sein.
- **Technologische Werkzeuge:** Nutze Softwaretools wie R in Kombination mit RShiny oder Python mit Dash. Alternativ kannst Du andere vergleichbare Python-Frameworks evaluieren, die für die Erstellung interaktiver Webanwendungen geeignet sind.
- **Softwarearchitektur:** Entwickle eine umfassende Software-Architektur mithilfe von UML-Sequenzdiagrammen, um die Interaktionen und Abläufe innerhalb der Anwendung präzise abzubilden.
- **Benutzeroberflächendesign:** Erstelle ein vorläufiges Design der Benutzeroberfläche mittels einer schematischen Zeichnung, um die Funktionalität und Einfachheit der Dateneingabe zu gewährleisten.
- **Bereitstellung und Distribution:** Plane strategische Maßnahmen zur effizienten Bereitstellung und Verteilung der Software, um eine einfache und konsistente Installation über verschiedene Plattformen hinweg sicherzustellen.
- **Skalierbarkeit und Anpassungsfähigkeit:** Entwickle Konzepte zur Skalierung der Software und zur Anpassung des Modells bei der Verfügbarkeit neuer Daten, um kontinuierliche Verbesserungen und Aktualisierungen zu unterstützen.
- **Datenschutz und Sicherheit:** Berücksichtige Datenschutzrichtlinien, Umsetzungsstrategien zur Sicherstellung von Datenintegrität, Vermeidung von Systemfehlern, Wahrung der Datenrelevanz sowie der IT-Sicherheit im gesamten Entwicklungsprozess.

Für die Abgabe ist ein **konzeptioneller Text (ca. 2 DIN-A4-Seiten)** zu erstellen und durch Grafiken zu ergänzen, sodass sämtliche Überlegungen beinhaltet sind. Das Konzept wird als **PDF-Datei** via pebblePad eingereicht.

Während des gesamten Prozesses gibt es im Rahmen der Online-Veranstaltungen die Möglichkeit, über Ideen und Entwürfe zu sprechen und sich Feedback einzuholen. In den Online-Veranstaltungen werden exemplarisch Arbeiten besprochen, die den Tutor:innen zuvor übermittelt wurden. Hier besteht für alle die Möglichkeit, sich einzubringen und vom Feedback der anderen zu lernen. **Es wird empfohlen, diese Kanäle zu nutzen, um Fehler zu vermeiden und Verbesserungen vorzunehmen. Erst danach sollen die Ergebnisse in der ersten Phase abgegeben werden.** Hier erfolgt ein abschließendes Feedback durch die Tutor:innen und die Arbeit in der zweiten Phase kann beginnen.

1.3.2. Erarbeitungs-/Reflexionsphase

In dieser Phase erfolgt die Implementierung der Klassifizierungssoftware basierend auf dem in der Konzeptionsphase entwickelten Plan unter Verwendung der ausgewählten Frameworks. Es ist zu erwarten, dass anfänglich Herausforderungen auftreten werden, die jedoch als integraler Bestandteil des Lernprozesses betrachtet werden sollten. Ausführliche Dokumentationen zu RShiny und Dash stehen auf den jeweiligen Dokumentationsseiten zur Verfügung und können hilfreich sein, ebenso wie zahlreiche Tutorials und Beispielprojekte, die zur Problemlösung beitragen können.

Das Frontend und das Backend sollen programmiert, getestet und dokumentiert werden. Folgende Schritte sind hierzu umzusetzen:

- Richte ein **offenes GitHub-Repository** ein, um den Code unter Nutzung von Versionskontrollmechanismen zu verwalten und zu speichern.
- Dokumentiere und kommentiere den Code gemäß bewährter Praktiken der Softwareentwicklung, um Lesbarkeit und Wartbarkeit sicherzustellen.
- Verpacke den Code in eine Einheit zur standardisierten Verfügbarkeit, z. B. mithilfe von Container-Technologien bzw. überlege Wege zur passenden Softwaredistribution.
- Führe eine Bewertung des trainierten Modells anhand standardmäßiger Leistungskennzahlen durch, um die Effizienz und Genauigkeit des Algorithmus zu quantifizieren.

Eine Erläuterung der Vorgehensweise wird als **Text eingereicht (1 DIN-A4-Seite)**. Die Datei soll den Link zum GitHub-Repository enthalten. Außerdem sollen die Schritte dieser Phase kurz beschrieben werden. Die Erklärung wird als **PDF-Datei** via pebblePad eingereicht.

Während des gesamten Prozesses gibt es im Rahmen der Online-Veranstaltungen und der weiteren Kanäle die Möglichkeit, über Ideen und Entwürfe zu sprechen und sich ausreichend Rückmeldung, Tipps und Hinweise zu holen. **Es wird empfohlen, diese Kanäle zu nutzen, um Fehler zu vermeiden und Verbesserungen vorzunehmen. Erst danach sollen die Ergebnisse in der zweiten Phase abgegeben werden.** Nach dem folgenden abschließenden Feedback durch die Tutor:innen wird in der dritten Phase an dem finalen Entwurf weitergearbeitet.

1.3.3. Finalisierungsphase

In der Abschlussphase besteht das Ziel darin, die Software auf Bugs zu prüfen und auf Stabilität zu testen. Bestimmte Elemente müssen eventuell noch einmal verbessert oder geändert werden. Du erstellst eine ausführliche **Zusammenfassung – full abstract (2 DIN-A4-Seiten)**, in der die Lösung der Aufgabe inhaltlich und konzeptionell beschrieben und der technische Ansatz kurz und übersichtlich dargestellt werden. Dabei sollte auch kurz auf die Qualität der Daten und die Ergebnisse der Analyse eingegangen werden. Diese Zusammenfassung sollte auch einen Ausblick darauf enthalten, wie das Verfahren weiter verbessert werden kann, sowie eine persönliche Reflexion über das Projekt: Was hast Du auf persönlicher Ebene gelernt? Auf welche Probleme bist Du gestoßen, und wie hast Du diese gelöst? Was sind Deine Problemlösungsstrategien für ähnliche anstehende Projekte?

Zusätzlich wird eine **Softwaredokumentation** geschrieben und ein **Walkthrough** aufgezeichnet. Die Video-Datei des Walkthroughs samt Dokumentation, die den Link zur **GitHub-Repository** enthält, wird in einen **ZIP-Ordner** verpackt und hochgeladen.

Auch in der Finalisierungsphase gibt es im Rahmen der Online-Veranstaltungen und der weiteren Kanäle die Möglichkeit, sich ausreichend Rückmeldung, Tipps und Hinweise zu holen, bevor das fertige Produkt final abgegeben wird. Es wird empfohlen, diese Kanäle zu nutzen, um Fehler zu vermeiden und Verbesserungen vorzunehmen.

2. BETREUUNGSPROZESS

Bei der Betreuung der Portfolios stehen grundsätzlich mehrere Kanäle offen. Die jeweilige Inanspruchnahme liegt dabei im eigenen Verantwortungsbereich. Die eigenständige Erarbeitung eines Produktes und die Befüllung der jeweiligen Portfolioteile ist dabei Teil der zu erbringenden Prüfungsleistung und fließt in die Gesamtbewertung mit ein.

Zum einen sieht die tutorielle Betreuung Feedbackschleifen zu den einzureichenden Portfolioteilen im Rahmen der Konzeptions- sowie der Erarbeitungs- und Reflexionsphase vor. Das Feedback erfolgt im Rahmen einer Einreichung des jeweiligen Portfolioteils. Des Weiteren werden regelmäßige Online-Veranstaltungen angeboten, in denen Gelegenheit besteht, mit den Tutor:innen Fragen zur Bearbeitung des Portfolios zu besprechen. Die Tutor:innen stehen zusätzlich für fachliche Rücksprachen sowie für formale und allgemeine Fragen zum Vorgehen bei der Portfoliobearbeitung zur Verfügung.

Technische Fragen zur Nutzung von PebblePad sind per Mail an das Prüfungsamt zu richten.

3. ZUSATZINFORMATIONEN ZUR BEWERTUNG

In die Bewertung des Portfolios fließen die folgenden Kriterien mit dem jeweils angegebenen Prozentsatz ein:

Bewertungskriterien	Erläuterungen	Gewichtung
Problemabgrenzung/Zielsetzung	*Erfassung des Problems *klare Problemabgrenzung/Zielsetzung *nachvollziehbares Konzept	10%
Methodik/Idee/Vorgehen	*angemessener Transfer von Theorien/Modellen *klare Angaben zur gewählten Methodik/zur gewählten Idee/zum gewählten Vorgehen	20%
Qualität der Umsetzung	*Qualität der Umsetzung und Dokumentation	40%
Kreativität/Richtigkeit	*Kreativität des Lösungsansatzes *umgesetzte Lösung erfüllt angestrebte Zielsetzung	20%
Formale Anforderungen	*Einhaltung der formalen Vorgaben	10%

Bei der Konzeption und Erstellung des Portfolios sollten die genannten Bewertungskriterien einschließlich der folgenden Erläuterungen berücksichtigt werden.

Problemabgrenzung/Zielsetzung: Fähigkeit, die Herausforderungen der Aufgabe zu verstehen und eine Strategie und Lösung zu entwerfen, die als Grundlage für die Entwicklung der Aufgabe dient.

Methodik/Idee/Vorgehen: Fähigkeit zur Umsetzung der geplanten Entwicklungsstrategie in eine konkrete Softwarestruktur mit genau definierten Modulen, Funktionen und deren Interaktion.

Qualität der Umsetzung: Trotz der Funktionalität ist es sehr wichtig, lesbaren Code zu erstellen, der von anderen leicht verstanden und geändert werden kann. Darüber hinaus spielt die Wahl der Programmierung, z. B. der implementierten Algorithmen und Werkzeuge sowie deren Test eine wichtige Rolle bei der Bewertung der Qualität der Implementierung.

Kreativität/Richtigkeit: Fähigkeit, kreative Lösungen für die identifizierten Herausforderungen zu finden, die entweder die Entwicklung der Aufgabe vereinfachen oder die Möglichkeit bieten, weitere Funktionalitäten, die nicht in der Aufgabenbeschreibung aufgeführt sind, einfacher zu implementieren

Formale Anforderungen: Das Folgende ist sowohl in den Code-Projekten (mit Verweis auf Code-Kommentare) als auch im erstellten Dokument zu beachten: Die Grammatik und die Syntax sollen fehlerfrei sein. Die Beschreibung des Inhalts muss gut strukturiert und für andere stets leicht lesbar sein. Lange verschachtelte Sätze sind zu vermeiden.

4. FORMALIA UND VORGABEN ZUR ABGABE

4.1. Bestandteile der Prüfungsleistung

Im Folgenden befindet sich eine Übersicht der Prüfungsleistung Portfolio mit seinen einzelnen Phasen, einzureichenden Einzelleistungen und Feedbackrunden im Überblick. Für die Erarbeitung der Portfolioteile im Rahmen der Prüfungsleistung wird eine Vorlage in PebblePad zur Verfügung gestellt. Die Vorlage ist Bestandteil dieser Prüfungsleistung.

Phase	Zwischenergebnis	Einzureichende Leistung
Konzeptionsphase	Portfolioteil 1	• Konzeptvorstellung in Textform (ca. 2 Seiten DIN A4) • UML-Sequenzdiagramm (im Konzept-PDF) • UI-Skizze (im Konzept-PDF)
Feedback		
Erarbeitungsphase/ Reflexionsphase	Portfolioteil 2	• Erläuterung zur Umsetzung in Textform (ca. 1 Seite DIN A4) • Link zur GitHub-Repository
Feedback		
Finalisierungsphase	Portfolioteil 3	• 2-seitige Zusammenfassung • Link zur GitHub-Repository • Software-Container (wenn die Größe es zulässt) • Softwaredokumentation • Video Walkthrough
Feedback + Note		

4.2. Formalia zur Abgabe digitaler Dateien

Konzeptionsphase

Empfohlene Hilfsmittel/Software zur Bearbeitung	Textverarbeitung, MS-Word u. a. oder Latex UML-Diagrammtool: draw.io oder vergleichbare
Zugelassene Dateiformate	.pdf
Dateigröße	möglichst gering/Preview
Weitere Formalien und Parameter	Dateien sind immer nach folgendem Muster zu benennen: Für die prüfungsleistungsrelevanten Abgaben auf PebblePad: Nachname-Vorname_Matrikelnummer_Kurs_P(hase)-1_A(bgabe) Beispiel: Musterperson-Manu_12345678_Print-Layout_P1_A

Erarbeitungs-/Reflexionsphase

Empfohlene Hilfsmittel/Software zur Bearbeitung	Textverarbeitung, MS-Word u. a. oder Latex Programmierung: R Studio für R oder VSCode/PyCharm für Python Containerisierung: Docker o. Ä.
Zugelassene Dateiformate	.pdf, .py, .R, .txt
Dateigröße	möglichst gering/Preview
Weitere Formalien und Parameter	Dateien sind immer nach folgendem Muster zu benennen: Für die prüfungsleistungsrelevanten Abgaben auf PebblePad: Nachname-Vorname_Matrikelnummer_Kurs_P(hase)-2_A(bgabe) Beispiel: Musterperson-Manu_12345678_Print-Layout_P2_A

Finalisierungsphase

Empfohlene Hilfsmittel/Software zur Bearbeitung	Textverarbeitung, MS-Word u. a. oder Latex Programmierung: R Studio für R oder VSCode/PyCharm für Python Containerisierung: Docker o. Ä. integrierter Screenrecorder des jeweiligen Betriebssystems
Zugelassene Dateiformate	.pdf, .py, .R, .mp4
Dateigröße	möglichst gering/Preview
Weitere Formalien und Parameter	

WICHTIG ist das Einfügen eines eigens für die Abgabe erstellten zip-Ordners (hierzu bitte Anleitung beachten). Um eine bessere Übersichtlichkeit zu gewährleisten, legt ihr hierzu bitte Unterverzeichnisse an.

Die Ordnerstruktur sieht dann folgendermaßen aus:

- Hauptverzeichnis (Benennung zip-Ordner) -> Benennung: Nachname-Vorname_Matrikelnummer_Kurs
 - Unterverzeichnis (Softwarecontainer) -> Benennung: „01-Software“
 - Unterverzeichnis (Dokumentation: Zusammenfassung+ DSoftwaredokumentation + Link zu GitHub-Repository) -> Benennung: „02-Dokumentation“
 - Unterverzeichnis (der Videowalkthrough) -> Benennung: „03-Walkthrough“
 - Unterverzeichnis (die Original-Datei(en), an denen ihr gearbeitet habt, sowie das fertige Druckabgabe-PDF) -> Benennung: „04-Reinzeichnung“

Dateien sind immer nach folgendem Muster zu benennen:

Für die prüfungsleistungsrelevanten Abgaben auf PebblePad:

Nachname-Vorname_Matrikelnummer_Kurs_P(hase)-3_A(bgabe)

Beispiel: Musterperson-Manu_12345678_Print-Layout_P3_A

4.3. Formalia für das Abstract

Umfang	2 Seiten Textteil
Papierformat	DIN A4
Seitenränder	Oben und unten 2cm; links 2cm; rechts 2cm
Schrifttyp	Allgemeiner Text – Arial 11Pkt; Überschriften – 12Pkt, Blocksatz
Zeilenabstand	1,5
Satz	Blocksatz und Silbentrennung
Fußnoten	Arial 10Pkt, Blocksatz
Absätze	Nach gedanklicher Gliederung – 6Pkt Abstand nach Zeilenumbruch
Eidesstattliche Erklärung	Die Abgabe der Eidesstattlichen Erklärung erfolgt in elektronischer Form über myCampus. Davor ist keine Einreichung der Prüfungsleistung möglich.

Bitte beachtet hierzu die Anleitung für das Einreichen eines Portfolios in myCampus.

Bei Fragen zur Abgabe des Portfolios wende Dich bitte per Mail an das Prüfungsamt.
Beachte bitte zusätzlich die Nutzungsanleitung zu PebblePad & Atlas!

Viel Erfolg beim Erstellen des Portfolios!